

DATABRICKS

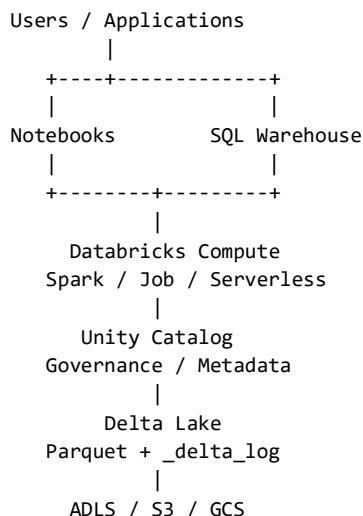
Optimized from the supplied Databricks study document

Interview lens: At 14+ years, interviewers test engineering judgment—not command memorization. Structure answers as: requirement → design → trade-off → implementation → failure mode → performance → cost → governance → measurement.

Area	What this optimized guide gives you
Architecture	Control Plane, Data Plane, compute choices, storage, Unity Catalog
Delta Lake	Physical model, _delta_log, ACID, DML, MERGE, CDC, SCD1/SCD2, time travel
Auto Loader	Incremental file ingestion, checkpointing, schema drift/evolution, rescue
Performance	Small files, pruning/skipping, joins, shuffle, skew, AQE, clustering
Enterprise	Medallion, idempotency, DQ/quarantine, metadata-driven pipelines, security
Interview	Scenario bank, senior answers, rapid-fire questions, final checklist

1. Databricks Architecture — Mental Model

The supplied document separates compute from storage and distinguishes Databricks management/orchestration from workload execution.



Component	Responsibility	Senior wording
Control Plane	Workspace, jobs, APIs, configuration, permissions, orchestration and lifecycle	Manage/orchestrate
Data Plane	Actual workload execution	Execute Spark/SQL workloads
Spark Compute	Distributed Python/SQL/Scala/PySpark processing	Use for complex ETL/data engineering
SQL Warehouse	SQL, BI, dashboards, analytics	Use for SQL-centric consumption
Serverless	Databricks manages much of the underlying compute infrastructure	Elastic + operationally simpler; not free
Unity Catalog	Governance, permissions, metadata, lineage, auditing	Govern access and traceability
Delta Lake	Transactional table/storage layer	Parquet data + transaction log
ADLS/S3/GCS	Physical cloud object storage	Storage remains separate from compute

Control Plane vs Data Plane

Control Plane	Data Plane
Workspace / UI	Spark execution
Job definitions	Driver
Cluster/workload configuration	Executors
Authentication / permissions	Reads/writes
APIs / orchestration	Joins, filters, aggregations
Metadata/control operations	SQL/Python/Scala workloads

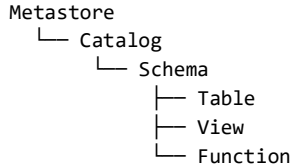
Interview trap: Do not say the Control Plane performs Spark transformations. The supplied material explicitly places actual computation on data-plane compute.

Compute choice

Compute	Best fit	Trade-off
All-purpose	Development, exploration, debugging, notebook work	Convenient, but persistent/shared compute can waste production cost
Job compute	Production workflow execution	Isolated and lifecycle-oriented; good production default
SQL Warehouse	BI/reporting/SQL analytics	Prefer over general Spark for SQL-centric workloads
Serverless	Elastic/managed workloads	Less infrastructure management; still incurs compute cost

2. Unity Catalog — Governance & Security

The supplied document uses the hierarchy Metastore → Catalog → Schema → Table/View/Function and emphasizes access control, discovery, lineage, auditing, storage credentials and external locations.



catalog.schema.table

Concept	Purpose
Metastore	Top-level governance container
Catalog	Logical data domain / top-level namespace
Schema	Namespace for tables, views and functions
Managed table	Databricks manages table storage location in managed storage
External table	Table references an explicitly supplied cloud storage location
Storage credential	Credential used for governed storage access
External location	Governed mapping between cloud path and credential
Lineage	Trace data from source through tables/views to consumers

```

CREATE EXTERNAL LOCATION bronze_location
URL 'abfss://bronze@storageaccount.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL my_credential);

```

```

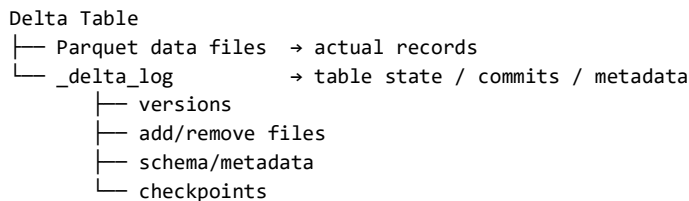
GRANT SELECT
ON TABLE clinical.ct_operations.studies
TO `data_analysts`;

```

Senior security answer: Use governed storage access, least privilege, enterprise identity/managed identity where applicable, secret management and network controls. Do not distribute unrestricted storage credentials through notebooks.

3. Delta Lake — Core Interview Topic

Memorize: Delta Lake = Parquet data files + _delta_log + transactional table management.



Capability	Purpose	Interview cue
ACID	Reliable transactional updates and consistent snapshots	Explain Atomicity/Consistency/Isolation/Durability
MERGE	Upsert, CDC and SCD patterns	Very common enterprise pattern
UPDATE/DELETE	Transactional mutations	Think replacement files + commit, not cell editing
Time travel	Historical table reads	Useful for audit/debug/recovery; not full DR
Schema enforcement	Protect target schema	Contract protection
Schema evolution	Controlled schema changes	Contract change, not the same as drift
CDF	Row-level change feed	Different purpose from time travel

OPTIMIZE	Improve physical file layout	Consumes compute; workload-driven
VACUUM	Remove eligible obsolete physical files	Retention-sensitive

Delta vs Parquet

Parquet	Delta Lake
Columnar file format	Table/storage layer
Stores columnar data	Uses Parquet for data files
No complete transactional table layer by itself	Adds <code>_delta_log</code> and table semantics
Good physical format	ACID, MERGE, DML, time travel, schema management

Physical lifecycle

```

Version 0
  ADD file-A.parquet
  |
Version 1  INSERT
  ADD file-B.parquet
  |
Version 2  UPDATE
  REMOVE old file
  ADD replacement file
  |
Version 3  DELETE
  REMOVE affected file from active snapshot
  |
Current table snapshot = transaction history + active files

```

Create / INSERT

```

CREATE TABLE sales (
  id BIGINT,
  name STRING,
  salary DOUBLE
) USING DELTA;

INSERT INTO sales VALUES
(1, 'A', 100.0),
(2, 'B', 200.0);

INSERT INTO sales
SELECT id, name, salary
FROM staging_sales;

```

UPDATE / DELETE

```

UPDATE sales
SET salary = 250.0
WHERE id = 2;

DELETE FROM sales
WHERE id = 2;

```

Internals: The supplied material explains UPDATE/DELETE conceptually as file replacement and transaction commits. Old files can remain physically until they become eligible for cleanup.

MERGE / Upsert

```

MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN
  UPDATE SET *

```

```
WHEN NOT MATCHED THEN
  INSERT *;
```

Senior rule: Make the source deterministic before MERGE. If multiple source rows match one business key, the result can be ambiguous or fail depending on the operation.

CDC

```
MERGE INTO target t
  USING cdc_source s
  ON t.id = s.id

  WHEN MATCHED AND s.operation = 'UPDATE'
    THEN UPDATE SET *

  WHEN MATCHED AND s.operation = 'DELETE'
    THEN DELETE

  WHEN NOT MATCHED AND s.operation = 'INSERT'
    THEN INSERT *;
```

4. SCD Type 1 & Type 2

SCD Type 1

Overwrite the old value when historical versions are not required.

```
MERGE INTO dim_customer t
  USING customer_updates s
  ON t.customer_id = s.customer_id
  WHEN MATCHED THEN UPDATE SET *
  WHEN NOT MATCHED THEN INSERT *;
```

SCD Type 2

customer_id	city	start_date	end_date	is_current
100	Pune	2026-01-01	2026-08-29	false
100	Mumbai	2026-08-29	NULL	true

Senior implementation sequence:

1. Deduplicate source by business key and latest valid change timestamp.
2. Identify current target row using business key + current flag.
3. Compare tracked attributes (direct comparison or normalized hash).
4. Expire the old row when a real change is detected.
5. Insert the new version as the current row.
6. Keep effective dates, current flag and audit columns deterministic.
7. Optimize the dimension based on actual MERGE/query access patterns.

Hash comparison

```
from pyspark.sql import functions as F

tracked = ["name", "city", "status"]

staged = updates.withColumn(
    "record_hash",
    F.sha2(
        F.concat_ws(
            "||",
            *[
```

```

        F.coalesce(F.col(c).cast("string"), F.lit("<NULL>"))
        for c in tracked
    ]
),
256
)
)

```

NULL trap: Normalize NULLs before hashing and keep the tracked-column order stable. A hash is only useful if the input representation is deterministic.

Deterministic deduplication

```

from pyspark.sql import functions as F
from pyspark.sql.window import Window

w = Window.partitionBy("customer_id").orderBy(
    F.col("modified_date").desc()
)

dedup_df = (
    source_df
    .withColumn("rn", F.row_number().over(w))
    .filter(F.col("rn") == 1)
    .drop("rn")
)

```

5. Time Travel, History, Schema & Maintenance

Time travel

```

SELECT * FROM sales VERSION AS OF 99;

SELECT *
FROM sales
TIMESTAMP AS OF '2026-08-28 10:00:00';

DESCRIBE HISTORY sales;

```

Recovery distinction: Time travel is useful for historical inspection and recovery when required files remain available. It is not a substitute for an enterprise backup/DR strategy.

VACUUM

```
VACUUM sales RETAIN 168 HOURS;
```

- VACUUM removes eligible obsolete physical files according to retention rules.
- It is not a query-acceleration command.
- Aggressive retention can remove files needed for historical reads.
- Retention must be aligned with recovery, time-travel, audit and concurrent-reader requirements.

Checkpoint

Delta checkpoints summarize transaction-log state so the table does not need to replay every historical JSON commit from the beginning.

```

_delta_log/
  000000...json
  000001...json
  ...
  000010...json
  000010.checkpoint.parquet

```

Schema enforcement vs evolution

Enforcement	Evolution
Protects against incompatible/unexpected writes	Allows an intentional schema change
Think: contract protection	Think: controlled contract change
Example: incompatible type should not silently pass	Example: additive nullable column

6. Delta Optimization & Spark Performance

Senior principle: Do not answer every performance question with 'OPTIMIZE'. First identify whether the bottleneck is scan, files, shuffle, skew, joins, aggregation, layout or compute.

Technique	Main purpose	Remember
Partitioning	Coarse data elimination	Avoid high-cardinality over-partitioning
Data skipping	Skip irrelevant files using file statistics	Not the same as partition pruning
OPTIMIZE	Compact/reorganize files	Costs compute
Z-ORDER	Improve data locality/data skipping	Choose columns from real query patterns
Liquid clustering	Flexible clustering for evolving workloads	Use when supported and justified
VACUUM	Physical obsolete-file cleanup	Retention-sensitive
AQE	Runtime Spark optimization	Not Delta maintenance
Broadcast join	Avoid shuffle of small side	Validate size/memory

Small-file problem

Example: $100,000 \times 1 \text{ MB files} = 100 \text{ GB}$. The issue is not only storage size; file discovery, metadata and task overhead can materially affect runtime.

```
OPTIMIZE sales;
```

Optimization cost: OPTIMIZE rewrites data. Running it too frequently can cost more than the performance benefit. Use file growth, file sizes, query frequency, SLA and cost to set the cadence.

Z-ORDER / clustering

```
OPTIMIZE sales
ZORDER BY (customer_id);
```

```
OPTIMIZE sales
ZORDER BY (customer_id, sale_date);
```

Better interview wording: Z-ORDER improves data locality so file-level statistics and data-skipping mechanisms can eliminate more irrelevant files for selective queries.

Partition pruning vs data skipping

Partition pruning	Data skipping
Eliminates whole partitions	Eliminates individual files
Coarse-grained	File-level / finer-grained
Driven by partition predicates	Uses file statistics and layout

Performance troubleshooting

```
Slow query
|
+--> Large scan? -----> filter / pruning / skipping / clustering
|
+--> Small files? -----> write pattern / OPTIMIZE
|
+--> Shuffle? -----> join / groupBy / repartition
|
```

```

+--> Skew? -----> AQE / salting / join strategy
|
+--> Bad join? -----> broadcast / filter early / join design
|
+--> Spill? -----> partition sizing / memory / reduce data
|
+--> Still slow? -----> compute / execution engine / workload design
|
+--> Measure -----> runtime + scan + cost

```

Spark UI / Query Profile

- Input and output size
- Shuffle read and write
- Exchange stages
- Task duration and task imbalance
- Skewed partitions
- Spill
- Join strategy
- Filter pushdown and partition pruning
- File count and file-size distribution

Broadcast join

```
from pyspark.sql.functions import broadcast
```

```

result = fact_df.join(
    broadcast(dim_df),
    "customer_id",
    "left"
)

```

Interview example: The supplied material uses a 5-TB fact table and a ~20-MB dimension as an intuitive broadcast case. Always validate the actual size, memory and workload before forcing a broadcast.

Why OPTIMIZE may not fix a slow query

- Large shuffle
- Data skew
- Poor join strategy
- Missing/late filtering
- Large aggregation
- Poor clustering/data locality
- Insufficient or inappropriate compute

14+ answer: Table maintenance and Spark execution tuning are separate layers. Diagnose both.

7. Auto Loader & Schema Drift

The supplied document describes Auto Loader as incremental cloud-file ingestion built on Structured Streaming. Its two core responsibilities are file discovery and state management.

```

Cloud Storage
|
Auto Loader
/      \
Discovery  State / Checkpoint
|
Structured Streaming

```

|
 Bronze Delta

Term	Meaning
Auto Loader	Incremental ingestion of new cloud files
checkpointLocation	Streaming processing state/progress
cloudFiles.schemaLocation	Schema inference/evolution tracking
Schema drift	Source schema changes, including unexpected/breaking changes
Schema evolution	Controlled acceptance of intentional schema changes
_rescued_data	Preserve data that cannot be safely mapped under configured schema behavior

Production pattern

from pyspark.sql import functions as F

```

df = (
    spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", "/mnt/schema/customer/")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("/mnt/landing/customer/")
        .withColumn("_ingestion_time", F.current_timestamp())
        .withColumn("_source_file", F.input_file_name())
)

(
    df.writeStream
        .format("delta")
        .option("checkpointLocation", "/mnt/checkpoint/customer/")
        .outputMode("append")
        .trigger(availableNow=True)
        .toTable("bronze.customer")
)

```

Critical distinction: schemaLocation answers 'what is the structure of incoming data?'; checkpointLocation answers 'what streaming state/progress do I have?' Never casually delete or move a production checkpoint.

Schema drift decision

```

Incoming file
|
Auto Loader
|
Schema detection
/      Expected      Changed
|
Process      Additive? ---> controlled evolution
              |
              Breaking? ----> validate / rescue / quarantine
              |
              Silver

```

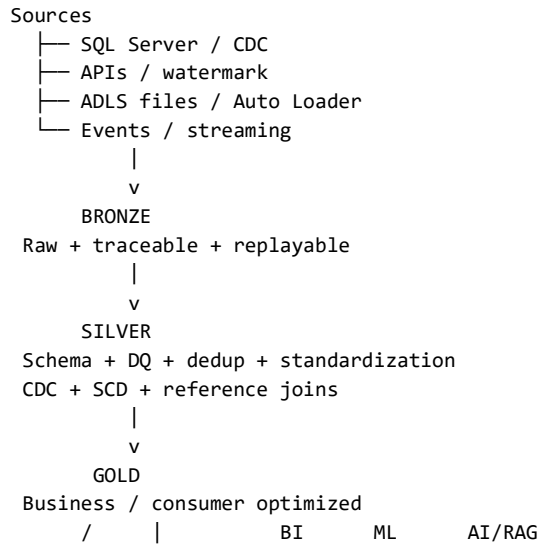
- Additive new nullable column: usually manageable with controlled evolution.
- Type change such as INT → STRING: treat as a breaking change.
- Monitor new columns, type changes, missing columns, null rates and rescued records.
- Preserve source filename, ingestion timestamp and batch/run identifiers.

Auto Loader vs COPY INTO

Auto Loader	COPY INTO
Incremental cloud-file ingestion	Batch-oriented incremental loading
Structured Streaming based	SQL ingestion pattern

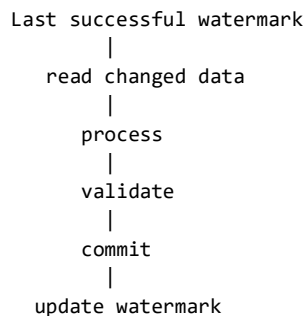
Strong fit for continuously arriving files	Good for simpler batch ingestion
Checkpoint + schema tracking	Ingestion tracking

8. Enterprise Lakehouse Pipeline Design



Layer	Purpose	Typical work
Bronze	Preserve source fidelity and traceability	Raw data, ingestion timestamp, source file, batch ID, source system
Silver	Engineering-quality data	Validation, typing, dedup, standardization, DQ, CDC, SCD, PII handling
Gold	Business-ready consumption	KPIs, aggregates, Customer 360, domain data products

Incremental processing



Senior signal: If 10 TB exists but only 50 GB changed, do not process 10 TB every night unless the business semantics require it. Use CDC, watermarking, Auto Loader, CDF or event-driven patterns where appropriate.

Idempotency

Running the same pipeline twice should not create duplicate business state.

```

MERGE INTO silver.customer t
USING staging_customer s
ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

```

Late-arriving data

- Separate event time from processing time.
- Define a business-approved late-arrival window.
- Use watermarking for applicable streaming scenarios.
- For batch, reprocess only affected partitions when possible.
- Keep the process deterministic and auditable.

```
df.withWatermark("event_time", "2 hours")
```

Data quality + quarantine

```

Incoming
|
Data Quality
/      PASS      FAIL
|          |
Silver    Quarantine
          |
        reason / batch_id
        source / timestamp

```

- NOT NULL / required-field checks
- Reference/domain validation
- Range checks
- Format checks
- Uniqueness / duplicate checks
- Source-target reconciliation
- Never silently discard invalid records

Metadata-driven architecture

```

metadata
source_system | source | load_type | key | target
SQLSERVER     | orders | CDC      | id  | silver.orders
ADLS          | cust  | FILE     | id  | bronze.customer
API           | study | WATERMARK | id  | silver.study

```

```

Generic framework
-> read metadata
-> select ingestion strategy
-> validate
-> write target
-> audit + metrics

```

9. Jobs, Workflows & Operational Design

```

Workflow
|
+--> Ingestion
|
+--> Data Quality
|
+--> Silver transform
|
+--> MERGE / SCD2
|
+--> Gold
|
+--> Maintenance

```

|
+--> Audit / notification

Control	14+ expectation
Dependencies	Model task dependencies explicitly; avoid one giant notebook
Parameters	Environment/source/run configuration instead of hard-coded values
Retries	Retry transient failures; don't blindly retry data-quality failures
Timeouts	Protect against hung workloads and SLA breaches
Idempotency	Safe rerun without duplicate business records
Observability	Run ID, batch ID, counts, duration, status, errors, freshness
Alerts	Failures, DQ thresholds, SLA breaches
Compute	Prefer isolated production job compute or suitable serverless option

Failure handling

```

Transient infrastructure failure
|
retry
|
success? ---- yes ---> continue
|
no
v
alert / incident

Data-quality failure
|
quarantine
|
no blind retry
|
remediation / replay

```

10. Security, Cost, CI/CD & Reliability

Security flow

```

Source
|
Secure network / private connectivity
|
Managed identity / service principal
|
Cloud storage
|
Databricks
|
Unity Catalog
|
RBAC / ABAC / governed access
|
Gold / BI / ML / Applications

```

- Use least privilege.
- Avoid hard-coded secrets in notebooks.
- Use enterprise secret management such as Key Vault where applicable.
- Separate environment permissions.
- Audit access and pipeline execution.

Cost optimization hierarchy

1. Process less data
|
2. Scan less data
|
3. Shuffle less data
|
4. Use the right join strategy
|
5. Right-size compute
|
6. Optimize table layout
|
7. Schedule maintenance intelligently
|
8. Measure runtime + scan + cost

Senior signal: The smallest cluster is not automatically the cheapest. Compare total workload cost and SLA. A larger cluster that finishes much faster can sometimes be more cost-effective.

CI/CD and testing

Area	Production approach
Git	Version all code/configuration
CI	Unit tests, linting, security checks, SQL/PySpark validation
Integration	Representative Delta test data
Promotion	Controlled Dev → UAT/Test → Prod deployment
Configuration	Environment-specific parameters
Data tests	Schema, nulls, uniqueness, reconciliation and business rules
Regression	Known scenarios and expected metrics
IaC	Use where enterprise standards require infrastructure as code

11. Senior Scenario Bank

Area	Question	Strong answer direction
Design	Design a 2-TB/day pipeline from SQL Server, APIs and ADLS.	CDC + watermark + Auto Loader; Bronze/Silver/Gold; UC; job compute; DQ; observability; CI/CD; cost.
Architecture	Why Bronze/Silver/Gold?	Traceability/replay → engineering quality → consumer optimization.
Compute	All-purpose vs job compute?	Interactive development vs isolated production workload.
Compute	When serverless?	Elasticity and reduced infrastructure management; not free.
Governance	How secure ADLS access?	UC + governed storage + enterprise identity + least privilege.
Delta	Why Delta instead of Parquet?	ACID, MERGE/DML, schema management, time travel and reliable table state.
Delta	Is Delta a file format?	No; table/storage layer using Parquet plus <code>_delta_log</code> .
Delta	What happens during UPDATE?	Replacement data files + transaction commit; not in-place cell editing.
Delta	OPTIMIZE vs VACUUM?	Layout/compaction vs obsolete physical-file cleanup.
Delta	Z-ORDER vs partitioning?	File locality/skipping vs coarse partition elimination.
Delta	How SCD2?	Dedup → detect change → expire old → insert new current → maintain effective dates.
Delta	How make SCD2 efficient?	Incremental input, deterministic source, hash comparison, current-row filtering, appropriate layout.

Delta	MERGE is slow?	Check source duplicates, target scan, file fragmentation, join/shuffle and clustering.
Delta	Bad update in production?	Inspect history/time travel; recover/reprocess; don't treat time travel as DR.
Auto Loader	New column appears?	Controlled schema evolution + validation + monitoring.
Auto Loader	INT becomes STRING?	Breaking change; validate and control the source-contract change.
Auto Loader	schemaLocation vs checkpoint?	Schema tracking vs streaming processing state.
Auto Loader	Why protect checkpoint?	It contains state required for correct resume/recovery.
Spark	5-TB query still slow after OPTIMIZE?	Inspect scan, shuffle, skew, joins, filters, clustering and compute.
Spark	One task much slower?	Likely skew; inspect distribution and join/groupBy behavior.
Spark	Large fact + small dimension?	Potential broadcast, after validating size/memory.
Spark	Why groupBy expensive?	Wide transformation and shuffle.
Cost	How reduce Databricks cost?	Process/scan/shuffle less + right-size compute + workload-driven maintenance.
Reliability	How safe reruns?	Idempotent keys, deterministic MERGE, checkpoints, run/batch IDs and bounded retries.
DQ	Bad Silver records?	Quarantine with reason, source, batch ID and timestamp.
Late data	Event arrives late?	Business late-arrival policy + event/processing time + targeted reprocessing.
Metadata	100 sources?	Metadata-driven framework instead of bespoke pipelines.
Operations	How monitor?	Freshness, duration, counts, DQ, latency, failures, alerts and cost.
Security	Secrets?	Managed identity/service principal + enterprise secret management.
CI/CD	Dev to Prod?	Versioned artifact + automated tests + environment config + controlled promotion.
DR	Is time travel enough?	No; build a separate DR/backup/recovery strategy.
Design	Always streaming?	No; choose batch/streaming/available-now according to SLA and source behavior.

12. Rapid-Fire Questions to Memorize

Term	One-line senior answer
Control Plane	Manage/orchestrate Databricks resources and metadata.
Data Plane	Execute actual workloads.
Serverless	Databricks-managed compute infrastructure.
All-purpose	Interactive development compute.
Job compute	Workload-specific production compute.
SQL Warehouse	SQL/BI/analytics compute.
Unity Catalog	Central governance, security, metadata and lineage.
Delta Lake	Transactional table/storage layer.
Parquet	Columnar file format.
_delta_log	Transaction history/table-state mechanism.
ACID	Atomicity, Consistency, Isolation, Durability.
MERGE	Conditional upsert/update/insert/delete.
Time travel	Read historical table state.
CDF	Capture row-level changes.
OPTIMIZE	Improve file layout/compaction.
Z-ORDER	Improve locality and data skipping.

Liquid clustering	Flexible clustering for changing access patterns.
VACUUM	Remove eligible obsolete physical files.
Partition pruning	Eliminate irrelevant partitions.
Data skipping	Eliminate irrelevant files.
AQE	Runtime adaptive Spark execution.
Shuffle	Redistribution of data across partitions.
Skew	Uneven partition sizes causing stragglers.
Broadcast join	Send a small join side to executors.
Auto Loader	Incremental cloud-file ingestion.
checkpointLocation	Streaming state/progress.
schemaLocation	Auto Loader schema tracking.
Schema drift	Source schema changes, including unexpected changes.
Schema evolution	Controlled acceptance of schema changes.
SCD1	Overwrite; no history.
SCD2	Preserve historical versions.
Idempotency	Rerun produces intended same business state.
Quarantine	Persist invalid records with diagnostics.
Metadata-driven	Configuration controls generic pipeline behavior.
Bronze	Raw, traceable, replayable.
Silver	Validated, standardized, deduplicated, business-rule-ready.
Gold	Consumer/business optimized.
SLA	Service-level target such as completion/freshness.
SLO	Reliability target used to manage service performance.
Cost optimization	Process less, scan less, shuffle less, right-size compute.

13. Model Interview Answers

30-second: Explain Delta Lake

Answer: Delta Lake is a table/storage layer on cloud object storage. It typically stores actual data as Parquet and maintains a `_delta_log` transaction log. Together they provide transactional semantics such as ACID, reliable MERGE/UPDATE/DELETE, schema management and time travel. Compute processes the data, Unity Catalog governs it, and ADLS/S3/GCS provides physical storage.

60-second: Auto Loader + schema drift

Answer: Auto Loader is Databricks' incremental file-ingestion framework for cloud storage. It handles scalable discovery and persists processing state through checkpoints. I keep a persistent `schemaLocation` for schema tracking and define an explicit evolution strategy. Additive changes can be evolved when approved; breaking changes such as type changes require validation and controlled deployment. I also preserve source-file metadata, quarantine bad records and monitor schema changes before promoting data to Silver.

90-second: Enterprise pipeline

Answer: I would design an incremental, metadata-driven lakehouse. Database sources would use CDC or watermarks, file sources Auto Loader, and event sources streaming where the SLA requires it. Bronze preserves source fidelity plus ingestion metadata for replay and audit. Silver handles schema validation, standardization, deterministic deduplication, data quality, CDC and SCD processing; invalid records go to quarantine. Gold is consumer-oriented for BI, ML and applications. Delta provides transactional storage and Unity Catalog provides governance, access, lineage and audit. Production workloads use job-oriented or suitable serverless compute, with idempotency, retries, monitoring and CI/CD. For performance I diagnose scan, shuffle, skew, joins and file layout before changing compute. For cost I minimize data processed/scanned, right-size compute and measure runtime, scan volume and cost before and after.

5-minute: 5-TB table is slow

8. Clarify workload and SLA; do not start by running OPTIMIZE.

9. Inspect query profile / EXPLAIN and Spark UI.
10. Measure input scan and output volume.
11. Check partition pruning and data skipping.
12. Check file count and file-size distribution.
13. Check joins, shuffle, exchange and aggregation.
14. Check skew and task imbalance; consider AQE/salting where appropriate.
15. Check whether a small dimension can be broadcast safely.
16. Evaluate partitioning and clustering based on actual query patterns.
17. Right-size compute only after the bottleneck is understood.
18. Measure before/after runtime, scan volume and cost.
19. Schedule maintenance based on workload, growth, SLA and retention—not habit.

14. Final 14+ Interview Checklist

- ☐ Explain Control Plane vs Data Plane without mixing responsibilities.
- ☐ Choose All-purpose vs Job compute vs SQL Warehouse vs Serverless with trade-offs.
- ☐ Explain Unity Catalog hierarchy and governed storage access.
- ☐ Explain Delta as Parquet + `_delta_log`.
- ☐ Explain INSERT / UPDATE / DELETE / MERGE at both SQL and conceptual levels.
- ☐ Explain ACID, checkpoints, time travel and VACUUM correctly.
- ☐ Implement SCD1 and explain a robust SCD2 design.
- ☐ Use deterministic deduplication before MERGE.
- ☐ Explain hash comparison and NULL normalization.
- ☐ Explain Auto Loader, schemaLocation, checkpoint and schema drift.
- ☐ Distinguish schema enforcement from schema evolution.
- ☐ Diagnose small files, scan, shuffle, skew, join strategy and spill.
- ☐ Distinguish partition pruning, data skipping and clustering.
- ☐ Design incremental and idempotent enterprise pipelines.
- ☐ Design DQ + quarantine + replay + reconciliation.
- ☐ Cover security, lineage, audit, CI/CD, monitoring, SLA/SLO and DR.
- ☐ Quantify optimization using runtime + scan + cost.